

System Maintenance Document

YESMAXX

System Maintenance Document EASL

System Architecture Overview

The Yes Maxx EASL system consists of modular components that together form a reproducible active-learning pipeline for bias evaluation.

Each module is self-contained, version-controlled, and can be replayed using deterministic seeds.

This architecture ensures comprehensive **auditability, reproducibility, and transparency** across all stages, from data ingestion to annotator assignment.

Core Modules

selector_cli.py	Batch formation and round orchestration	selector_decisions_*.csv
scorer.py	Uncertainty scoring (Proxy & Main)	uncertainty_score, reason, scorer
selection.py	Policy enforcement and balancing	Balanced batch selection
build_baselines.py	Compute cohort bias means	cohort_baselines.json
schema_utils.py	Schema normalisation and data validation	Clean joined DataFrames
build_profiles.py	Annotator calibration, topic affinity	annotator_profiles.csv, centroids
assign_utils.py	Per-item × annotator feature scoring	Feature matrix (M-scores)
overlap_assign.py	Profile-aware assignment and redundancy control	Annotator CSVs, manifest.json

Module Summaries

selector_cli.py - Active Selection Orchestrator

Role:

Forms each round's candidate batch.

Process:

Loads prompts, runs, and responses excludes previously selected items invokes scorer.py to generate uncertainty signals applies per-stream quotas and per-prompt caps.

A deterministic, seeded ordering guarantees reproducibility. In hybrid mode, capacity is divided between Proxy and Main scorers, and a small recheck subset is included to support inter-rater reliability (IRR) assessment.

Output:

selector_decisions_*.csv containing full lineage of items, scores, and selection reasons.

scorer.py - Uncertainty Scoring

Role:

Computes both exploration (Proxy) and exploitation (Main) uncertainty measures.

Proxy (Exploration):

$$u_{\text{proxy}}(i) = 1 - \text{mean}_{\{j \neq i\}}(\text{cosine}(x_i, x_j))$$

This approach selects lexically diverse responses to promote coverage of new regions within the response space.

Main :

$$y'_{\{i,a\}} = y_{\{i,a\}} - \delta\mu[a,s]$$

$$u_{\text{main}}(i) = \text{var}(y'_{\{i,a\}}) + \varepsilon$$

This method prioritizes items exhibiting the highest disagreement following calibration adjustment.

Variance cascades from item to prompt to global.

Batch-wise min-max scaling maintains score comparability across items.

Output:

uncertainty_score, reason, and scorer labels.

selection.py - Policy and Balancing

Role:

Transforms ranked scores into a balanced batch.

Process:

The process sorts by uncertainty, resolves ties deterministically, applies per-stream quotas and per-prompt caps, and ensures minimum coverage for each bias stream.

In hybrid mode, proxy and main selections are merged, and a small recheck subset is incorporated for IRR tracking.

build_baselines.py - Cohort Baselines

Role:

The produced reference bias means used for annotator calibration.

Process:

Aggregates de-duplicated annotations to compute bias means at four levels: global, stream, template, and model.

If a given stratum is too sparse, the process defaults hierarchically to a broader level, such as from stream to global.

Output: cohort_baselines.json containing all hierarchical means.

schema_utils.py - Data Normalisation and Contracts

Role:

Guarantees schema stability and referential integrity across all joins and merges.

Process:

The process maps synonym field names, enforces primary and foreign key constraints, converts timestamps to ISO-8601 UTC format, standardizes categorical codes to lowercase, removes duplicates, and validates each join prior to downstream computation.

This approach prevents header drift and malformed merges between rounds.

build_profiles.py - Annotator Profiles

Role:

Learns annotator calibration offsets and topic affinities.

Formulas:

$$\delta\mu[a] = \text{mean}_k(y_{\{a,k\}} - \mu_{\text{item}[k]})$$

$$\delta\mu[a,s] = \text{mean}_{\{k \in s\}}(y_{\{a,k\}} - \mu_{\text{item}[k]})$$

Process:

Computes both global and per-stream calibration deltas.

Builds exposure counts and TF-IDF topic centroids for each annotator:

$$c_a = (1 / N_a) * \sum_k x_k$$

$$c_a = c_a / ||c_a||$$

Profiles are refreshed after each round, utilizing all labels up to round N-1, to prevent bias leakage.

Output:

annotator_profiles.csv and centroid vectors for topic affinity analysis.

assign_utils.py - Feature Matrix for Allocation

Role:

Generates the multi-feature matching matrix $M(i,a)$ used to pair items with annotators.

Features:

- Topic Fit: $T = \text{cosine}(x_i, c_a)$
- Stream Affinity: Encourages coverage of underrepresented bias streams.
- Calibration Gap: $C = |\delta\mu[a, \text{stream}(i)]|$
- Uncertainty: $U = \text{item-level uncertainty from scorer.py}$

Composite Scoring Function:

$$M(i,a) = \alpha T + \beta A + \gamma C + \delta U - \text{penalties}$$

Default weights: $\alpha = 0.25$, $\beta = 0.10$, $\gamma = 0.30$, $\delta = 0.30$.

Each feature is normalized to the range $[0,1]$. Penalties are lambda-weighted cooldown terms that prevent redundant assignments or overloading of a single annotator.

overlap_assign.py - Profile-Aware Assignment

Role:

Allocates items to annotators under fairness and overlap constraints.

Pair Scoring:

$$S(i,a,b) = M(i,a) + M(i,b) - \text{duo_pen}(a,b)$$

Duo Penalty:

$$\text{duo_pen}(a,b) = \eta * [\text{pair_count}[a,b] / (1 + \text{pair_total}[a] + \text{pair_total}[b])]$$

Controlled overlap parameter p maintains redundancy for agreement metrics while minimizing repeated pairings.

Output: manifest.json and per-annotator CSVs recording achieved overlap, top-duo statistics, and final assignments.

Core Formulae Reference

Formula	Equation	Reasoning
Cosine similarity	$\text{cosine}(u,v) = (u \cdot v) / (u v)$	Vector similarity metric
Proxy uncertainty	$u_{\text{proxy}}(i) = 1 - \text{mean}_{\{j \neq i\}}(\text{cosine}(x_i, x_j))$	Exploration metric favouring lexical diversity
Main uncertainty	$y'_{\{i,a\}} = y_{\{i,a\}} - \delta\mu[a,s]$ $u_{\text{main}}(i) = \text{var}(y'_{\{i,a\}}) + \varepsilon$	Exploitation via calibrated disagreement
Calibration deltas	$\delta\mu[a] = \text{mean}_k(y_{\{a,k\}} - \mu_{\text{item}}[k])$ $\delta\mu[a,s] = \text{mean}_{\{k \in s\}}(y_{\{a,k\}} - \mu_{\text{item}}[k])$	Offset correction per annotator / stream
Topic centroid	$c_a = (1 / N_a) * \sum_k x_k$ $c_a = c_a / c_a $	Mean topic vector per annotator
Composite score	$M(i,a) = \alpha T + \beta A + \gamma C + \delta U - \text{penalties}$	Weighted multi-feature match
Pair overlap score	$S(i,a,b) = M(i,a) + M(i,b) - \text{duo_pen}(a,b)$	Two-rater allocation with penalty
Duo penalty	$\text{duo_pen}(a,b) = \eta * [\text{pair_count}[a,b] / (1 + \text{pair_total}[a] + \text{pair_total}[b])]$	Controls repeated pairings
Min-max normalisation	$\text{norm}(z) = (z - z_{\text{min}}) / (z_{\text{max}} - z_{\text{min}} + \varepsilon)$	Keeps batch metrics comparable
Penalties	λ -weighted cooldown + load factors	Prevents redundancy and annotator overload

Maintenance Guidelines

Rebuild cadence

- Recompute profiles and baselines after every annotation round ($N-1$).
- Run `schema_utils.validate()` before each selection round.
- Maintain a fixed selector seed (`seed_run`) to preserve deterministic replayability.

Storage and lineage

- All outputs are versioned by round under `/data/batches/round_N/`.
- Historical input files (`prompts.csv`, `runs.csv`, `responses.csv`, `annotations_*.csv`) must never be overwritten they should only be appended with timestamps.

Integrity checks

- Verify referential integrity of keys (`response_id`, `annotator_id`).
- Review the overlap manifest for achieved vs. target overlap (p).
- Periodically inspect calibration drift using `build_profiles.py` deltas.

Reproducibility

Executing the same modules with identical seeds and inputs yields identical `selector_decisions` and assignment files.

This guarantees full auditability and repeatability of every round.

System Maintenance Document - Annotation UI

Purpose & Scope

This guide provides instructions for operating, maintaining, and supporting the Annotation UI within the EASL round-based bias evaluation pipeline. It addresses system architecture, dependencies, configuration, data models, operational procedures, security, troubleshooting, and backup and restoration processes. Algorithmic details related to selection and profiling are documented separately in the Technical Methods document.

System Role in the EASL Loop (High-Level)

The Annotation UI serves as the **human-in-the-loop** component responsible for the following functions:

- Presents prompt to model response items to annotators.
- Collects scalar bias ratings (1-5) plus notes/metadata.
- Persists labels with stable IDs, timestamps, and round awareness.
- Exports analysis-ready CSVs and SQLite snapshots to feed the next round’s selector and profiling jobs.

Key exclusions: The UI does not compute selection scores or generate profiles. It only reads assignments produced by the selector and records annotation labels.

Architecture & Components

Layered Architecture

Frontend (UI)	Interactive scoring, navigation, and feedback	Streamlit
Logic Layer	Reads selector_decisions CSV, creates per-user assignment queues, manages pagination and label writes	Python, hashlib, st.session_state
Database Layer	Stores annotations, users, timestamps, bias scores, and audit metadata	SQLite

Data Flow (Workflow Diagram)

Load selector_decisions CSV to Build per-user assignment to Annotator Login to Scoring & Save to SQLite Storage to Export CSV/Snapshot

Key Modules & Responsibilities

Frontend	Streamlit widgets	Sliders (1-5), notes text box, Next/Back, progress meter, export button
Data Loading	pandas	Reads and validates CSVs (encoding-aware)
Database	SQLite + SQL	Local persistent storage without external dependencies
State Mgmt	st.session_state	Retains assignment queue and current index across sessions
Integrity	hashlib.md5	Detects upstream CSV changes (content checksum surrogate: filename + file size)
Export	pandas.to_csv + st.download_button	One-click export of per-round annotations

Current Implementation Binding

This section specifies that the maintenance guide applies to the current Streamlit application deployment.

Code & Environment

- Framework: Streamlit Data: pandas DB: sqlite3 (stdlib)
- Paths: DATA_PATH = "selector_decisions_main2_3.csv" DB_PATH = "annotations.db"
- Page: st.set_page_config(page_title="YesMaxx Annotation Workspace", layout="wide")

Annotator Roster & Roles

- Valid annotators (hard-coded): Xin, Yong, Mahir, Saqif, Ammar. Input is normalised with .capitalize() and must match exactly.
- Admin is gated by annotator == "Xin". Admin tools shown: Clear All Annotations Download annotations_export.csv.

Assignment Policy

- On first load or when the CSV signature changes, the app builds a fixed 20-item assignment per annotator using `random.shuffle` on IDs 1..N and slicing into 5 buckets.
- Assignments are stored in `st.session_state["assignments"]`. If the Streamlit server restarts, assignments are regenerated and may differ from previous assignments however, labels that have already been saved remain in the database.
- Current record = `assigned_ids[idx]`, with zero-based row access via `responses_df.iloc[assigned_id-1]`.

CSV Detection & Columns

Signature: `md5(f"{file_name}_{file_size}")`. Note: this detects renames/size changes, but not content-only edits with identical name and size.

Ensured columns:

- `response_id`: created 1..N if absent.
- `run_id`: detected by fuzzy header match defaults to "run_003" if missing.
- `response_text`: maps to any header containing "text" (lowercased). If none, app stops.
- Prompt display: uses `prompt_text_x` if present, else `prompt_text`.

De-duplication & Saves

- Prevents duplicates per annotator via lookup on (`response_id`, `annotator`) before INSERT.
- DB schema allows duplicates the app enforces prevention. Time stored as `datetime.now()` (local time).

Export & Metrics

- Export button label: "Download annotations.csv" to save `annotations_export.csv`.
- Sidebar metrics: All annotations (distinct `response_id`), My annotations (for current user).

Navigation

- Buttons: Previous, Next, Restart. After submit, auto-advance and `st.rerun()`.

Data Contracts & Schemas

Actual App Schema (SQLite annotations)

The current deployment uses a minimal schema created at startup:

- CREATE TABLE IF NOT EXISTS annotations (
- id INTEGER PRIMARY KEY AUTOINCREMENT,
- response_id TEXT,
- run_id TEXT,
- annotator TEXT,
- bias_score INTEGER,
- notes TEXT,
- timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
-)

Notes for maintainers

- **Uniqueness** is enforced in the **app logic** (not by a DB constraint): one row per (response_id, annotator).
- timestamp is written using datetime.now() (local time) even though the column has a DEFAULT CURRENT_TIMESTAMP.
- File: annotations.db at project root.

Export CSV

- Header: id,response_id,run_id,annotator,bias_score,notes,timestamp
- File name: annotations_export.csv (via sidebar download button).

Required Input (Selector Decisions)

selector_decisions_main_n*.csv in the working directory.

Minimum required columns

- response_id (optional to auto-generated 1.N)
- run_id (optional to auto-detected by header substring else defaults to run_003)
- response_text (required mapped from any header containing text)
- prompt_text_x or prompt_text (for display only)
- Other columns (e.g., model_id, stream, uncertainty_score) may exist but are not required by the current app.

The UI reads but does not modify the selector CSV.

Annotation Output (Per Round)

DB: annotations.db (authoritative store for the app)

CSV Export: annotations_export.csv (download from sidebar)

Round awareness is implicit (by file name and timing). If you need explicit round_index later, add a column at export time in a separate tool no change to this app is necessary.

Required Input (Selector Decisions)

File pattern: selector_decisions_*.csv

Minimum required columns:

- response_id (string, FK to responses)
- run_id (string, FK to runs)
- prompt_id (string)
- model_id (string)
- uncertainty_score (float)
- reason (enum: proxy/main/recheck)
- scorer (string)
- stream (enum: cultural,economic,ethnic,gender,political, ...)
- selected_at (UTC ISO 8601)
- seed (int)
- assignment_type (enum: primary,overlap)

The UI does not modify this file. It only reads to build assignment queues.

Annotation Output (Per Round)

CSV: annotations_main_r{N}.csv

- response_id (string)
- prompt_id (string)
- model_id (string)
- annotator_id (string)
- stream (enum)
- round_index (int)
- bias_score (int 1-5)
- confidence (optional float 0-1)
- notes (optional string)
- timestamp_utc (UTC ISO 8601)
- app_version (string)
- source_md5 (md5 of selector file used for this session)

SQLite snapshot: annotations_r{N}_YYYYMMDD.sqlite

- Tables: annotations, annotators, sessions, audit_log

DDL

- CREATE TABLE IF NOT EXISTS annotations (
 - id INTEGER PRIMARY KEY AUTOINCREMENT,
 - response_id TEXT NOT NULL,
 - prompt_id TEXT NOT NULL,
 - model_id TEXT NOT NULL,
 - annotator_id TEXT NOT NULL,
 - stream TEXT NOT NULL,
 - round_index INTEGER NOT NULL,
 - bias_score INTEGER NOT NULL CHECK (bias_score BETWEEN 1 AND 5),
 - confidence REAL,
 - notes TEXT,
 - timestamp_utc TEXT NOT NULL,
 - app_version TEXT,
 - source_md5 TEXT NOT NULL
-)
- CREATE TABLE IF NOT EXISTS annotators (
 - annotator_id TEXT PRIMARY KEY,
 - display_name TEXT,
 - role TEXT CHECK (role IN ('admin', 'annotator')) DEFAULT 'annotator',
 - created_utc TEXT
-)
- CREATE TABLE IF NOT EXISTS sessions (
 - session_id TEXT PRIMARY KEY,
 - annotator_id TEXT NOT NULL,
 - created_utc TEXT NOT NULL,
 - last_seen_utc TEXT,
 - FOREIGN KEY (annotator_id) REFERENCES annotators(annotator_id)
-)
- CREATE TABLE IF NOT EXISTS audit_log (
 - id INTEGER PRIMARY KEY AUTOINCREMENT,
 - event TEXT NOT NULL,
 - actor TEXT,
 - details TEXT,
 - created_utc TEXT NOT NULL
-)
- CREATE INDEX IF NOT EXISTS idx_annotations_keys ON annotations(response_id, annotator_id)

Dependencies & Environment

Software

- Python: 3.9 - 3.11
- Packages: streamlit, pandas, numpy, python-dateutil, pytz (optional), sqlalchemy (optional), typer (optional CLI), stdlib sqlite3, hashlib

requirements.txt

- streamlit>=1.33
- pandas>=2.1
- numpy>=1.26
- python-dateutil>=2.8
- pytz>=2024.1
- SQLAlchemy>=2.0
- typer>=0.12

Directory Layout

- project_root/
- data/
- selector/
- selector_decisions_main_rN.csv
- exports/
- annotations_main_rN.csv
- annotations_rN_YYYYMMDD.sqlite
- snapshots/
- config/
- .env
- config.toml
- app/
- app.py
- db.py
- io.py
- integrity.py

Configuration (.env or config.toml)

- ANNOTATION_DB_PATH=./data/exports/annotations_current.sqlite
- SELECTOR_DIR=./data/selector
- EXPORT_DIR=./data/exports
- DEFAULT_ASSIGNMENT_SIZE=20
- OVERLAP_RATE=0.10 # 10% IRR targeting
- ROUND_INDEX=1

- APP_SECRET=change_me

All configuration reads must be logged to audit_log at startup.

Installation & Startup

- **Create environment**
 - python -m venv .venv && source .venv/bin/activate
 - pip install -r requirements.txt
- **Prepare folders**
 - Create data/selector and place selector_decisions_*.csv inside.
 - Ensure data/exports is writable.
- **Initialize DB**
 - On first run, the app auto-creates the SQLite schema.
 - Seed admin user: add one row to annotators with role='admin'.
- **Run app**
 - streamlit run app/app.py
- **Access**
 - Open the provided local URL and log in with your annotator ID.

Operations Runbook

Pre-Round Checklist

- Place selector_decisions_main_n*.csv in the working directory.
- Ensure the file really changed if you intend to rebuild assignments. The app detects changes via name + size content-only changes with identical name/size will not trigger a rebuild.

During Round

- Do not restart the Streamlit server mid-round assignments live in st.session_state and will be reshuffled if the server restarts (saved labels are safe).
- Use the sidebar metrics to monitor total vs. per-user progress.

Close Round

- Use the Download annotations.csv button to produce annotations_export.csv.
- Backup annotations.db alongside the CSV.
- Optionally compute visuals (coverage, histograms) in your analysis notebook.

Disaster Recovery

- From CSV: Rebuild DB by importing the authoritative annotations_main_rN.csv.
- From Snapshot: Restore annotations to N_YYYYMMDD.sqlite into ANNOTATION_DB_PATH.

Security, Access & Privacy

- Accounts/Roles: Annotators are identified by name input. The current roster is hard-coded: Xin, Yong, Mahir, Saqif, Ammar.
- Admin: The app exposes admin tools only when annotator == "Xin" (no password in this build). Admin can clear all annotations and trigger CSV export.
- PII: Store only the annotator name avoids sensitive content in notes.
- Audit: Actions are observable via DB contents and server logs there is no separate audit table in this build.
- Data Handling: The SQLite file annotations. dB and exported CSVs may contain sensitive notes handle per institutional policy.

Data Integrity & Quality Gates

- Input validation: Header match, required columns non-null, enum value sets enforced (stream, assignment_type).
- Score validation: bias_score $\in \{1 \text{ to } 5\}$ confidence $\in [0, 1]$ if present.
- Duplicate prevention: Unique (response_id, annotator_id) on conflict, keep latest timestamp_utc and log to audit_log.
- Time: Enforce UTC local display may be converted for UX only.

Backup, Retention & Archival

- Per-round snapshots: Keep annotations N_YYYYMMDD.sqlite for at least 12 months (configurable).
- CSV Exports: Treat annotations_main_rN.csv as the authoritative interchange format.
- Rotation: Weekly offsite copy of data/exports.
- Restore test: Quarterly verification by restoring a random round snapshot.

Logging & Monitoring

- Application logs: Streamlit server logs (stderr/stdout) captured to logs/app_YYYYMMDD.log.
- Audit log: Persistent table records admin actions and exports.

- Metrics (optional): Simple counts-labels/hour, active annotators, overlap IRR coverage-emitted as CSV.

Performance & Concurrency

- Target scale: 5-30 annotators concurrently.
- Latency: Label save < 100 ms locally export < 2 s for ≤50k rows.
- Indexes: idx_annotations_keys improves query performance for export and de-dup.

Maintenance Tasks

- Before each round: Validate inputs, bump ROUND_INDEX, rotate previous snapshot, announce quotas.
- Weekly: Backup exports, review logs for errors, verify disk space.
- Quarterly: Restore drill from snapshot, run schema health check.

Change Management & Versioning

- SemVer for the UI app (app_version stored with each label).
- Migrations: If schema changes, bump minor version and include migration script. Always backward-exportable to the canonical CSV schema.

Troubleshooting Guide

- Symptom: Annotations reshuffled after a restart.
- Cause: Assignments are stored in st.session_state, which resets on server restart.
- Fix: Avoid restarts during rounds if it happens, continue-duplicates are prevented per annotator.
- Symptom: Export button does nothing.
- Cause: Browser download blocked or Streamlit permission issue.
- Fix: Allow downloads check server logs.
- Symptom: “CSV changed” logic didn’t trigger on content edit.
- Cause: Signature uses only file name + size.
- Fix: Rename the file or adjust the CSV so size changes before reloading then assignments will rebuild.
- Symptom: Duplicate label warning.
- Cause: Same annotator tried to re-label the same response_id.
- Fix: Proceed to Next edits require admin DB tooling.
- Symptom: SQLite lock on macOS/Windows.
- Cause: Another process (e.g., antivirus) holding the file.
- Fix: Exclude project folder from realtime scans keep a single writer.

System Impact

- Reliability: Persistent state and planned overlap on high-uncertainty items improve IRR.
- Data integrity: De-duplication and timestamping produce clean, auditable trails.
- Transparency: Centralized storage, clear schemas, and export buttons simplify verification.
- Research-readiness: Outputs are structured for visualization and downstream model auditing.

System Maintenance Document - Prompts-n8n

Purpose

This document outlines maintenance, monitoring, and update procedures for the n8n-based prompt generation pipeline within the EASL bias evaluation system. The pipeline automates controlled creation, validation, and storage of bias-probing prompts for multiple language models, supporting reproducibility, lineage tracking, and adherence to safety constraints.

System Overview

The Prompts-n8n subsystem serves as the entry point for the EASL workflow. It integrates Google Sheets, functioning as the prompt registry, with large language model (LLM) APIs such as OpenAI, Claude, and Gemini to generate bias-specific prompts across multiple streams.

The subsystem enforces strict schema validation, reproducible family assignment, and automated storage of generated prompts with comprehensive lineage metadata.

Core functions:

- Batch initialization (Manual Trigger)
- Fetching candidate PromptIDs
- Cleaning, deduplication, and stream normalization
- Deterministic family assignment per bias stream
- Safe LLM prompt generation with uniqueness checks
- Structured output parsing and validation
- Final logging and Google Sheet append

Dependencies and Environment

Component	Description	Maintenance Notes
n8n	Workflow orchestration engine	Update quarterly ensure latest n8n Docker image or desktop build is used.
Google Sheets API	Cloud storage backend for prompts	Verify API credentials every 90 days refresh OAuth tokens as required.
OpenAI / Claude API	Model endpoints for prompt generation	Rotate API keys monthly update model versions (e.g., GPT-4.1, Claude 3.5) when verified stable.

JavaScript Functions (in Code nodes)	Logic for normalization, seeding, validation	Backup node code on repository change review after n8n version upgrades.
Prompt Catalog Sheet	Master registry for all generated prompts	Maintain one sheet per bias stream use deduplication keys (PromptID, family).
Local Backups	CSV export of prompt history	Weekly export to /data/backups/prompts_YYYYMMDD.csv.

Data Flow and Key Artifacts

Inputs:

- Candidate rows from Google Sheets: PromptID, bias_stream, status.
- Fixed configuration JSON defining stream-family mappings.

Outputs:

- Validated prompt records appended to Google Sheets with columns:
- PromptID, bias_stream, family, prompt_text, createdAt, safety_tag.
- Parser logs and error JSON for audit review (exported weekly).

Intermediate artifacts:

- Temporary LLM JSON output: { "prompt_text": "...", "safety_tag": "..." }
- Internal audit file: /n8n_logs/prompt_generation_YYYYMMDD.json

Maintenance Tasks

Task	Frequency	Responsible	Description
Node Health Check	Weekly	Maintainer	Open workflow in n8n to ensure no “Node missing” errors and all credentials are green.
Google Sheets API Test	Weekly	Maintainer	Trigger manual batch to confirm append operation success and dedup guard triggered correctly.
LLM Endpoint Validation	Monthly	Engineer	Test each model node with sample prompt check latency and JSON compliance.
Error Log Review	Weekly	Maintainer	Inspect /n8n_logs/*.json address parsing or safety errors.
Prompt Catalog Backup	Weekly	Maintainer	Export latest Google Sheets to CSV.
Token & Key Rotation	Monthly	Admin	Replace API keys for OpenAI/Claude refresh Google API credentials.
Version Review	Quarterly	Admin	Review n8n changelog and update to the latest minor release after the sandbox test.

Failure Modes and Recovery Procedures

Issue	Likely Cause	Resolution Steps
Workflow fails at Google Sheets Append	Expired credentials	Re-authenticate Google credentials in n8n settings.
Duplicate prompts inserted	Dedup key mismatch	Verify key formula (PromptID, family) in "Insert Prompts" node.
Malformed JSON from LLM	Model output drift	Review "Structured Output Parser" node logic adjust regex/schema enforcement.
Loop stuck / timeout	Unbounded iteration or LLM retry	Add maxIterations parameter or use "Error Handler" node.
Safety tags missing	LLM skipped required field	Force safety_tag fallback = "sensitive_topic_eval".
Prompt family imbalance	Wrong seed or mapping	Re-run normalization map and verify family count distribution.

Logging & Audit

- Log Location: /n8n_logs/ (local or mounted volume)
- Schema: Each run logs timestamp, PromptID, bias_stream, family, model, latency, and error (if any).
- Retention: Keep logs for at least 6 months.
- Audit Export: JSON + CSV summary automatically emailed (if configured) or manually exported.

Version Control & Reproducibility

Formula for deterministic family assignment:

`seed = sum(unicode(codepoints(PromptID)))`

`family_index = (seed * 7 + 13) mod len(families_for_stream)`

- Fixed mappings: Streams and allowed families are defined in `/config/family_map.json`.
- Change control: Any edit to family mapping or safety policy must be versioned (v1.1, v1.2, etc.) and logged in `CHANGELOG.md`.

Limitations & Future Improvements

- Near-duplicate detection: Currently uses text matching and LLM self-check add an embedding-based cosine similarity filter for stronger enforcement.
- Token budget control: Replace prompt-instruction length enforcement with programmatic token counting.
- Auto health alerts: Future n8n feature-add Slack or email alerts on node failure.

System Maintenance Document - Response-n8n

Purpose

This document outlines the maintenance, monitoring, and update procedures for the n8n-based response generation pipeline. The pipeline distributes a standardized prompt set to multiple large language model (LLM) APIs, standardizes outputs, assigns stable identifiers, measures latency, and logs all artifacts into Runs and Responses tables for subsequent selection, profiling, and analysis.

System Overview

The Responses-n8n subsystem serves as the execution engine, converting registered prompts into comparable model outputs while maintaining complete data lineage.

Core functions

- Batch start (manual trigger or scheduled run).
- Read a fixed prompt slice from the Prompt registry.
- Distribute the identical prompt list to each enabled model branch.
- Standardize outputs, including encoding, whitespace, and JSON structure.
- Assign run_id and response_id deterministically.
- Record start/end timestamps and compute latency.
- Append records to the Runs and Responses sheets and update the prompt status for each model.

Active model branches

- GPT-4.1 Mini
- Gemini 2.5 Flash
- DeepSeek R1

Dependencies & Environment

n8n	Orchestration for nodes and branching	Update quarterly test in sandbox before production.
Google Sheets API	Storage for Prompts / Runs / Responses	Re-auth every 90 days verify scopes after n8n upgrades.

LLM APIs	OpenAI, Google AI Studio, DeepSeek endpoints	Rotate API keys monthly pin model versions by label (e.g., gpt-4.1-mini-YYYYMM).
Code nodes (JS)	Output standardiser, ID/latency calculators	Version in repo review on n8n updates.
Config	Stream enable/disable, branch toggles, sheet IDs	Keep in /config/responses.json audit changes in CHANGELOG.md.

Reference environment variables / config

- PROMPTS_SHEET_ID=...
- RUNS_SHEET_ID=...
- RESPONSES_SHEET_ID=...
- ENABLED_MODELS=["gpt41mini","gemini25flash","deepseekr1"]
- ID_SALT="replace_me"
- RETRY_MAX=2
- TIMEOUT_MS=60000

Data Flow & Key Artefacts

Flow

The data flow consists of the following steps: trigger, fetch prompt slice, branch per model, call API, standardize text, tag identifiers and timestamps, write Runs row, write Responses row, update prompt status per model, and emit latency metrics.

Inputs

Prompts Sheet (authoritative): prompt_id, bias_stream, family, prompt_text, status_[model].

Outputs

- **Runs Sheet (append-only)**
 - Columns: run_id, model_id, batch_label, prompt_count, started_at_utc, finished_at_utc, latency_ms_p50, latency_ms_p95, errors, version_tag.
- **Responses Sheet (append-only)**
 - Columns: response_id, run_id, prompt_id, model_id, response_text_std, started_at_utc, finished_at_utc, latency_ms, safety_flag, status, hash_source.

Intermediates

- Raw LLM JSON objects, including text, safety tags, and metadata, are held in memory.
- Node logs are generated for each execution and stored in the /n8n_logs/responses_YYYYMMDD.json directory.

Identifier policy

- `run_id = hash(model_id | batch_label | started_at_utc | ID_SALT)`
- `response_id = hash(prompt_id | model_id | started_at_utc | ID_SALT)`
The hash function used is SHA-256 with hexadecimal truncation to 16 characters. No personally identifiable information (PII) is included.

Standardization rules

- Normalize to UTF-8 collapse repeated whitespace strip control chars.
- Enforce a configurable maximum length for responses. Store the full text in the sheet without truncation if storage quotas permit.
- Record the `safety_flag` when provided by the model otherwise, set the value to unknown.

Latency

Per response: `latency_ms = finished_at_utc - started_at_utc` For each run, compute the 50th and 95th percentile latency values across the batch.ch.

Maintenance Tasks

Node health check	Weekly	Maintainer	Open workflow verify all nodes are connected credentials green.
Sheets connectivity test	Weekly	Maintainer	Dry-run 1 prompt confirm append to Runs/Responses and prompt status update.
API quota & key rotation	Monthly	Admin	Rotate keys validate each model branch with smoke test.
Latency review	Each batch	Engineer	Check p95 spikes investigate slow endpoints/timeouts.

Error log review	Weekly	Maintainer	Inspect /n8n_logs/* triage parser/HTTP errors.
Config drift audit	Monthly	Admin	Compare /config/responses.json against production update CHANGELOG.
Version bump	Quarterly	Admin	Pin model versions tag version_tag in Runs.
Backup Sheets	Weekly	Maintainer	Export Runs/Responses to CSV: /data/backups/responses_YYYYMMDD.csv.

Failure Modes & Recovery Procedures

HTTP 401/403 to LLM	Expired/invalid API key	Rotate key re-test node re-run failed prompts only.
429 rate limits	Concurrency too high	Lower node concurrency add exponential backoff enable RETRY_MAX.
Long latency / timeouts	Service degradation	Increase TIMEOUT_MS queue retry if persistent, disable branch and notify stakeholders.
Malformed JSON / missing text	Model output drift	Harden parser add fallback extraction log status=parser_error.

Duplicate responses	Retry without idempotence	Ensure response_id is deterministic de-dupe by (prompt_id,model_id).
Sheet append failure	Sheets API quota/file lock	Backoff + retry write to local buffer (/n8n_spool/) then replay.
Prompt status not updating	Column rename/drift	Re-map column in node re-apply status patch for affected rows.
Hash collision (rare)	Short truncation	Increase hex length to 24 re-generate response_id for conflicted row document in audit.

Replay policy

- Idempotent replays utilize the same run_id and response_id inputs.
- When inputs differ (e.g., model version bumped), generate a new run_id and new response_id, and keep both rows.

Logging & Audit

- Location: /n8n_logs/ (mounted volume or server disk).
- Per-run record: timestamp, batch_label, models enabled, prompt count, success/failure counts, latency_ms_p50/p95.
- Per-error record: node name, prompt_id, model_id, HTTP status, parser error message, retry count.
- Retention: 6 months minimum.
- Exports: Weekly JSON to CSV summaries optional email/Slack alert on error rate > 5%.

Version Control & Reproducibility

- Determinism is ensured through stable prompt ordering, a fixed ID salt, and deterministic response_id generation from prompt_id, model_id, started_at_utc, and ID_SALT.
- Version tags: Store version_tag = responses-n8n@vMAJOR.MINOR + model_version_map in Runs.
- Change control: Any edits to standardiser, ID policy, or sheet schemas must be recorded in CHANGELOG.md Reproducible runs are achieved by re-executing the same prompts with identical model versions and ID salt, resulting in identical IDs and comparable latencies, except for network variance. network variance aside.

Limitations & Future Improvements

- Near-duplicate detection: Add an embedding-based cosine filter to flag semantically identical responses across models/runs.
- Adaptive throttling: Auto-tune per-branch concurrency using rolling p95 latency.
- Health alerts: Integrate Slack/email for node failures or error-rate thresholds.
- Schema contracts: Migrate Sheets to a managed DB (e.g., Postgres) with constraints and upsets for stronger integrity at scale.
- Implement optional regular expression-based redaction to remove personally identifiable information before logging responses to shared storage.

Github Link –

Name	Link	Description
Annotation UI Github Link	https://github.com/xxxxxinnnnn/streamlit-annotation	Annotation UI github link
Annotation UI Link	https://app-annotation-test.streamlit.app/	Annotation UI website link
Selector EASL	https://github.com/daiyan20mahir-netizen/selector_easl	EASL algorithm